

1 3D Vision

Camera Model

Pinhole Camera: $x' = f \frac{x}{z}$, $y' = f \frac{y}{z}$ (focal length f)
aperture big - blur; aperture small - dark

Camera and Lense: $z' = f + z_0$, $f = \frac{R}{2(n-1)}$

specific "in focus" distance - depth of field
camera parameters: intrinsics & extrinsics
 k, l : pixel/m $\alpha = f \cdot k$, $\beta = f \cdot l$

convert between Euclidian & Homogeneous coordinates

$$P' = \begin{bmatrix} \alpha x + c_x z \\ \beta y + c_y z \\ z \end{bmatrix} = \begin{bmatrix} \alpha & -\alpha \cot \theta & c_x & 0 \\ 0 & \beta / \sin \theta & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$MP = K[I \ 0]P = K[I \ 0] \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}_{4 \times 4} P_w = K[R \ T]P_w$$

$$R = R_x(\alpha)R_y(\beta)R_z(\gamma), \text{ usually } \theta = \pi/2$$

$$M = [m_1, m_2, m_3]^T, \left(\frac{m_1 P_w}{m_2 P_w}, \frac{m_2 P_w}{m_3 P_w} \right) \text{ in } E$$

Depth Images

Depth Image: a 2.5D representation
can't recover (x, y, z) or point-to-point dist without K
backprojection: given K , $(u, v) = (\alpha \frac{x}{z} + c_x, \beta \frac{y}{z} + c_y)$
Stereo sensors: $u - u' = \frac{B \cdot f}{z}$ = disparity
Active stereo: replace one camera with a projector

Voxel Grid: regular occupancy grid in 3D, easy to index and run 3D conv, but memory/compute is $O(n^3)$
+: regular structure, compatible with conv nets
-: surface location is coarse, upsampling is hard, voxelization loses detail

Mesh

graph {vertex, edge}

Triangle Mesh: $V = \{v_i\} \subset \mathbb{R}^3$, $E \subset V \times V$, $F \subset V \times V \times V$

stores explicit surface + topology; more compact than dense voxel grid, but irregular and harder for CNN-like processing

Point Cloud

point cloud = surface + sampling

Uniform sampling: triangle v_1, v_2, v_3
 $x = v_3 + a_1(v_1 - v_3) + a_2(v_2 - v_3)$, $a_1, a_2 \in [0, 1]$
+: easiest; -: irregularly spaced sampling

Farthest Point Sampling (FPS): NP-hard, greedy iteratively select

Distance metrics:

$$\text{Chamfer: } \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \sum_{y \in S_2} \min_{x \in S_1} \|x - y\|_2$$

$$\text{Earth Mover's: } \min_{\phi: S_1 \rightarrow S_2} \sum_{x \in S_1} \|x - \phi(x)\|_2$$

CD - insensitive to sampling; EMD - sensitive

Implicit Field

Signed Distance Function: $F = \text{dist } 2 \text{ surface}$

2D Marching Square: extract zero iso-surface threshold; binary; T/F; contour; linear interpolation

Implicit shape: represent shape by a func on space
 $f(x) = 0$: surface; < 0 : inside; > 0 : outside

other choices: unsigned distance / occupancy network

SDF: collision check; continuous geometry storage query any 3D point (not storing only surface samples)

3D Marching Cube: 3D version of marching square ambiguity $\rightarrow$ holes; larger local context / better tables

Mesh $\leftrightarrow$ SDF: sample points around mesh, compute nearest-surface distance, assign sign
zero iso-surface can be converted back to mesh by marching cube

3D Deep Learning

DeepSDF: learn continuous SDF with neural network
sample more points near the zero level set if final output uses marching cube

Eikonal equation: $\|\nabla f(x)\|_2 = 1$

single shape: overfit one network; multiple shapes: add latent code

NeRF: for novel view synthesis, map $(x, d) \rightarrow (\sigma, c)$
 σ : density/attenuation c : emitted color d : viewing dir
ray $r(t) = o + td$, render by volume integration along the ray

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i), T_i = \prod_{j < i} (1 - \alpha_j), C = \sum_i T_i \alpha_i c_i$$

3D Gaussian Splatting (3DGS): parameterize scene by sparse anisotropic 3D Gaussians
projected 3D Gaussian becomes 2D Gaussian; same volume rendering idea, but explicit and fast
parameterize only where density is nonzero

PointNet: unordered $\rightarrow$ permutation invariant NN
shared MLP on each point + symmetric pooling (max)
robust to resolution/noise; weakness: no local context

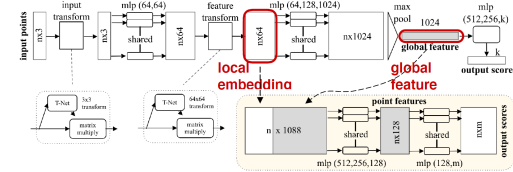


Figure 1: PointNet Implementation

PointNet++: hierarchical local PointNet
sample centers by FPS, group neighbors, shared MLP in local region, then maxpool
downsample by grouping; upsample by interpolation + skip-link

similar to local conv, but all points in one neighborhood are treated isotropically

Ball Query: collect local neighborhood within radius ball size and max queried points are hyperparameters
relative coordinates $x - x_c$ help extract translation-invariant local geometry feature

Set Abstraction: FPS + grouping + local PointNet
hierarchical feature learning with local translation invariance + permutation invariance

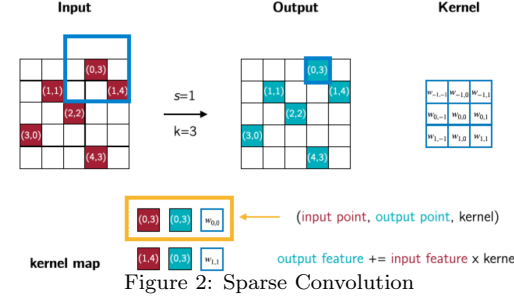
Segmentation in PointNet++: 3D interpolation + unit PointNet + skip link concatenation
for new points, interpolate features from nearest neighbors with inverse-distance weights

Voxel Networks: 3D CNN on volumetric data

easy extension of 2D conv, but dense 3D kernels are expensive and voxelization loses information

Sparse Conv: voxelize space, hash only active voxels
+: efficient on large scenes, similar inductive bias to 2D conv

-: quantization / discretization error, resolution limit



Sparse Conv vs Point Cloud Nets:

sparse conv: spatially anisotropic kernels, efficient indexing, good for large scenes
point cloud nets: higher resolution, easier for quick use, but FPS / ball query can be slow

2 Sequential Models

RNN

an internal state updated as the sequence is processed

$$h_t = f_W(h_{t-1}, x_t), y_t = f_{W_h}(h_t)$$

recurrence formula - same func & para each time step

Vanilla RNN: a single hidden vector h

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t), y_t = W_{hy}h_t$$

$$\text{gradient: } \frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}, \frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_0}{\partial W}$$

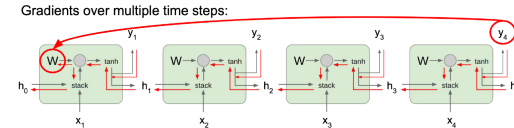


Figure 3: Vanilla RNN Gradient Flow

truncated backpropagation: run through chunks of seq
sequence length: ΔT , only learn shorter relationship
input $\rightarrow$ (tokenizer) $\rightarrow$ embedding $\rightarrow$ hidden $\rightarrow$ output

Sampling: greedy(determine) / weighted(wrong)

$$\text{Exhaustive Search: } P(y|x) = \prod_{t=1}^T P(y_t | y_1, \dots, y_{t-1}, x)$$

Beam Search: keep k most probable sequence (efficient)

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

$\tanh'(W_{hh}h_{t-1} + W_{hx}x_t)$ always < 1 : vanish gradients
more dependent on long-term memory, not near effects
 W_{hh} largest singularity > 1 : explode; < 1 : vanish
exploding - gradient clipping (scale)
vanishing - change RNN architecture

Long-Short Term Memory (LSTM)

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}, 4 \text{ gates}$$

$$c_t = f \odot c_{t-1} + i \odot g, h_t = o \odot \tanh(c_t)$$

Input-whether to write to cell; Forget-whether to erase
Output-how much to reveal; Gate-how much to write
only elementwise multiplication, no matrix mul by W
doesn't guarantee no exploding/vanishing

Gated Recurrent Unit (GRU): : other variant

Image Caption Using RNN

$$b_v = W_{hi}[CNN_{\theta_c}(I)]$$

$$h_t = f(W_{ht}x_t + W_{hh}h_{t-1} + b_h + 1(t=1) \odot b_v)$$

special token: <END> (finish)

small dataset: nearest neighbor (text/caption retrieval)

3 Attention & Transformer

Attention

Sequence to Sequence:

Encoder: $h_t = f_W x_t, h_{t-1}$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1})$

train: y correct input; wrong output cross-entropy loss

test: y_t input the output of y_{t-1}

problem: input sequence through fixed size c

alignment scores: $e_{t,i} = f_{att}(s_{t-1}, h_i)$ (linear layer)

attention weights: $\sum_i a_{t,i} = 1$

context vector: $c_t = \sum_i a_{t,i} h_i$

all differentiable, no supervision on attention weights

Query vecs & Data vecs transformed into Output vecs

each query attend to all data vec and give 1 output vec

inputs: query vec - $q[D_X]$ / data vec - $X[N_X \times D_X]$

computation: similarities - $e[N_X]$ $e_i = q \cdot X_i / \sqrt{D_X}$

(scaled) / attention weights - $a = \text{softmax}(e)[N_X]$ /

output vec - $y = \sum_i a_i X_i[D_X]$

permutation equivariant ($F(\sigma(X)) = \sigma(F(X))$)

Position embedding: a fixed vector of index

Rotary Positional Embedding: encode relative pos

$$(R(\theta_i)q_i)^T (R(\phi_j)k_j) = q_i^T R(\phi_j - \theta_i)k_j$$

Causal mask: prevent looking ahead while training

Multi-headed self attention: parallel H heads

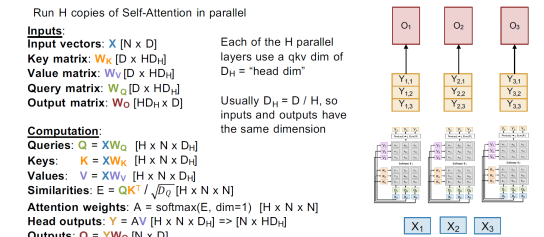


Figure 4: Multiheaded Self-Attention Layer

QKV projection - QK similarity - V weighting - output
projection: compute $O(N^2)$, memory $O(N^2)$

flash attention - memory $O(N)$ (2+3 the same time)

Transformers

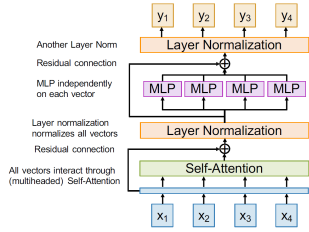


Figure 5: Transformer Block

normalization per token, each submodule gets its own $x \leftarrow x + f(x)$ - keep activation in controlled range
 Original(213M) - GPT-2(1.5B) - GPT-3(175B)
 Pre-Norm: Move normalization inside residual
 QK-Norm: Normalize keys and queries
 SwiGLU: Different MLP architecture
 Mixture of Experts (MoE): Learn E different MLPs, use A $\neq$ E of them per token. Massively increase params, modest increase to compute cost.

Visual Transformers

ViT: input - break into patches - flatten - transformer
 low inductive bias, better on large data

4 Generative Model

Generative Models

discriminative - learn $p(y|x)$ / generative - learn $p(x)$
 conditional generative model: $p(x|y) = \frac{p(y|x)p(x)}{p(y)}$
 model ambiguity - model $p(x|y)$ for many outputs

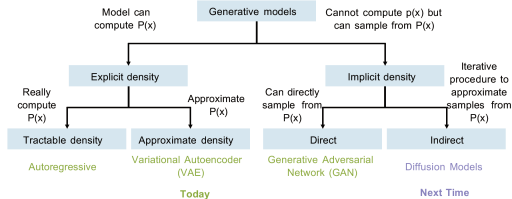


Figure 6: Taxonomy of Generative Models

Autoregressive Models

$W^* = \arg \max_W \sum_i \log f(x^{(i)}, W)$ loss function
 for sequence x : $p(x) = \prod_{t=1}^T p(x_t | x_1, x_2, \dots, x_{t-1})$
 image as sequence of subpixels, use RNN/Transformer
 too expensive - models as sequence of tiles

Variational Autoencoders (VAE)

image x , z is latent factors used to generate x

Probalistic Autoencoder: $p_{\theta^*}(x|z^{(i)}), z^{(i)} \sim p_{\theta^*}(z)$
 maximize $p_{\theta}(x) = \int p(z)p_{\theta}(x|z)dz$
 intractable: Monte Carlo estimation (high variance)
 $p(z)$: prior density; $p_{\theta}(z|x)$: posterior density
 variational approximation: learn $q_{\phi}(z|x)$ (decoder)

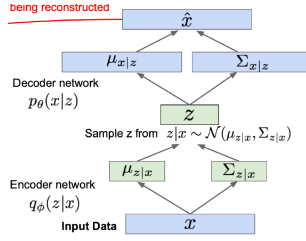


Figure 7: Variational Autoencoder

loss: $\mathcal{L} = -\log p_{\theta, \phi}(x)$, maximize $\log p_{\theta}(x^{(i)}) = \mathbf{E}_z[\log p_{\theta}(x^{(i)}|z)] - D_{KL}(q_{\phi}(z|x^{(i)})||p(z)) + D_{KL}(q_{\phi}(z|x^{(i)})||p_{\theta}(z|x^{(i)}))$
 in practice: $\Sigma_{z|x} = \mathbf{I}$

- Generative model and goal (x: data; z: latent variable)
 $p_{\theta}(x) = \int p_{\theta}(x|z)p(z) dz = \int p_{\theta}(x|z)p(z) dz$
- VAE introduces a variational posterior $q_{\phi}(z|x)$, then
 $p_{\theta}(x) = \int q_{\phi}(z|x)p_{\theta}(x|z)p(z) dz = \mathbf{E}_{q_{\phi}(z|x)} \left[\frac{p_{\theta}(x|z)p(z)}{q_{\phi}(z|x)} \right]$
- Jensen's inequality: for a random variable X , $\mathbf{E}[f(X)] \geq f(\mathbf{E}[X])$ if f is concave.
 $\log p_{\theta}(x) = \log \mathbf{E}_{q_{\phi}(z|x)} \left[\frac{p_{\theta}(x|z)p(z)}{q_{\phi}(z|x)} \right] \geq \mathbf{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x|z)p(z)}{q_{\phi}(z|x)} \right]$
 $= \mathbf{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x)||p(z))$ (ELBO)
- Finally, VAE maximizes

Figure 8: VAE in a Nutshell

Generative Adversial Networks(GAN)

not modeling any explicit density function
 minimax obj func: $\min_{\theta_g} \max_{\theta_d} [E_{x \sim p_{data}} \log D_{\theta_d}(x) + E_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))]$
 discriminator(θ_d) max obj, generator(θ_g) min obj
 generator-unsampling network with fractionally-strided convolutions; discriminator-convolutional network
 Evaluation: nearest neighbors / user study / mode drop and mode collapse

$FID(r, g) = ||\mu_r - \mu_g||_2^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$
 lower FID - smaller distance between synthetic & real

Diffusion Models

limitations of VAE: single Gaussian latent code
 hierarchical latent-variable model / Markov chain
 forward process gradually adds Gaussian noise until data becomes nearly standard normal
 reverse process learns to denoise step by step

Forward: $q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$
 let $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$
 closed form: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$

Reverse denoising: learn $p_{\theta}(x_{t-1}|x_t)$
 true reverse posterior is Gaussian; network predicts its mean or the noise

DDPM: training add known noise, then predict noise
 $\mathcal{L}_{DDPM} = \mathbf{E}_{x_0, \epsilon, t} ||\epsilon - \epsilon_{\theta}(x_t, t)||_2^2$
 sampling: start from $x_T \sim \mathcal{N}(0, I)$, iteratively denoise to x_0

can view diffusion as hierarchical VAE / score matching / solving an SDE

+: stable, high quality; -: many denoising steps, slow sampling

Latent Diffusion Models

Why latent: pixel-space diffusion is expensive
 compress image to latent $z \in \mathbb{R}^{H/D \times W/D \times C}$
 $D = 8$, $C = 16$, so $256 \times 256 \times 3 \rightarrow 32 \times 32 \times 16$

LDM pipeline: VAE encoder/decoder + diffusion in latent space
 train autoencoder, freeze encoder, add noise in latent space, denoise latent, decode to image
 modern pipelines often use VAE + GAN + diffusion
 VAE gives compression, GAN sharpens decoder output, diffusion models the latent distribution

Conditional diffusion: same denoising idea, but condition on text / image / control signal
 most common: cross-attention with text embeddings
 Stable Diffusion: pretrained text encoder + latent diffusion + decoder

Diffusion Transformer (DiT): use standard transformer blocks as denoiser
 inject timestep by scale / shift modulation; inject text or image by cross-attention / joint attention

5 Large Models II

Video Transformers

video = image + time

Joint space-time attention: for T frames and N tokens per frame, full attention cost $O((TN)^2)$
 captures all pairwise interactions but is expensive

Divided space-time attention: do temporal and spatial attention separately, cost $O(NT^2 + TN^2)$
 main efficiency idea: factorize attention instead of attending jointly over all tokens

Video Swin Transformer: restrict attention to local windows, then shift windows
 cheaper, good local motion modeling

MViT: multiscale vision transformer
 reduce token count with pooling / hierarchy while increasing channel dimension
 two broad strategies for efficient video: restrict attention range or reduce number of tokens

Foundation Models

Families:

Language	Classification	Chaining	LM + Vision	And More!
ELMo BERT GPT T5	CLIP CoCa	Image Description + LM LMs + CLIP Visual Programming	LLaVA FlanVLM GPT Gemini Qwen Molmo & PixMo	Segment Anything Whisper Dalle Stable Diffusion Imagen

Figure 9: Foundation Models

CLIP: learn concepts without labels (self-supervised)

Contrastive learning: learn aligned image-text embedding space
 image encoder+text encoder, similarity by dot product

$L_{i \rightarrow t} = -\frac{1}{B} \sum_i \log \frac{\exp(v_i^T t_i / \tau)}{\sum_j \exp(v_i^T t_j / \tau)}$
 usually add the symmetric text→image loss too

Usage: zero-shot classification by prompt engineering
 create text vector for each category, compare image embedding with all prompt embeddings

linear probe on frozen CLIP features is also strong
 CoCa: add a generative objective
 +: open-vocabulary/reusable encoder/strong semantics
 -: global image-text matching not enforce event struc
 CLIP may know the right objects but still confuse who did what to whom
 language compositionality and argument roles are weak

CLIP-Event: generate event-aware hard negatives / role-swapped negatives
 helps distinguish same entities with different roles, but hard negatives alone can still teach shortcuts

Chain foundation models:

VidIL: Language Models with Image Descriptors are Strong Few-Shot Video-Language Learners
 instead of training a full video-language model, chain frozen image-language models and a frozen LLM through text

Pipeline: sample frames → objects / events / attributes / frame captions / subtitle
 → temporal-aware few-shot prompt → LLM output

Temporal prompts: use markers like First / Then / Finally (temporal order markers)
 +: no video pretraining, few-shot capable, strong baseline for captioning / QA / event prediction

VS. Flamingo: compose existing models through language instead of training an integrated VLM

LLaVA: vision encoder(CLIP)+projector+autoregressive LLM

project visual features into language model token space, then decode text autoregressively
 CLIP encode - strong visual semantics; penultimate-layer features are often better than final pooled output

Training recipe: first align visual features to the LLM, then do instruction tuning
 inherits the autoregressive nature of LLMs, so naturally supports dialogue / description / QA

Flamingo: frozen vision encoder + frozen LM + newly inserted cross-attention layers
 LM attends to visual tokens through cross-attention instead of collapsing everything into one vector
 +: strong zero-shot / few-shot multimodal in-context learning
 keeps more visual detail than pure CLIP-style global embeddings

LLaVA = projector into autoregressive LLM
 Flamingo = frozen LM that reads visual tokens through cross-attention

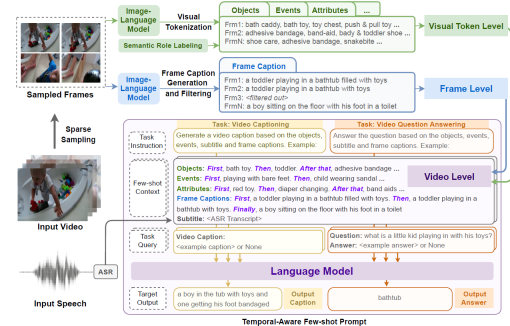


Figure 10: Large Models